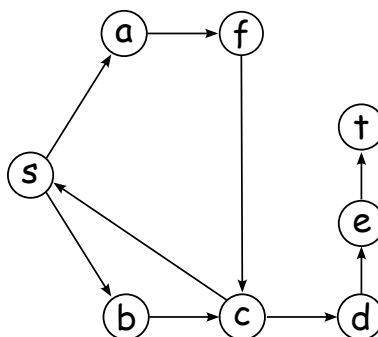


COMP364 Quiz 1

*Note: All search algorithms should use **graph** search (**not tree** search), unless stated otherwise. All outputs of search algorithms must be written using the format discussed in class.*

Question 1. BFS (10 points)

Trace the execution of the BFS algorithm on the graph below. You must write your output in the format discussed in class.



Solution.

index	state	parent	depth	status
0	s	—	0	x
1	a	0	1	x
2	b	0	1	x
3	f	1	2	x
4	c	2	2	x
5	d	4	3	x
6	e	5	4	x
7	t	6	5	t

Path found: $s \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow t$

Question 2. DFS (10 points)

Trace the execution of the DFS algorithm on the same graph (above). You must write your output in the format discussed in class.

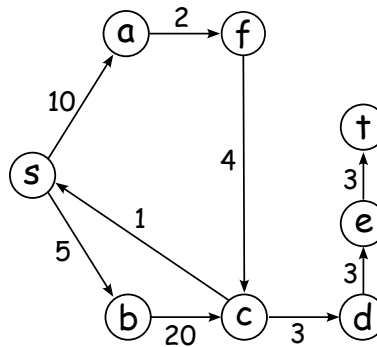
Solution.

index	state	parent	depth	status
0	s	—	0	x
1	a	0	1	x
2	b	0	1	
3	f	1	2	x
4	c	3	3	x
5	d	4	4	x
6	e	5	5	x
7	t	6	6	t

Path found: $s \rightarrow a \rightarrow f \rightarrow c \rightarrow d \rightarrow e \rightarrow t$

Question 3. Uniform cost search (also known as Dijkstra's algorithm) (10 points)

Trace the execution of the uniform cost search algorithm on the weighted graph below. You must write your output in the format discussed in class.



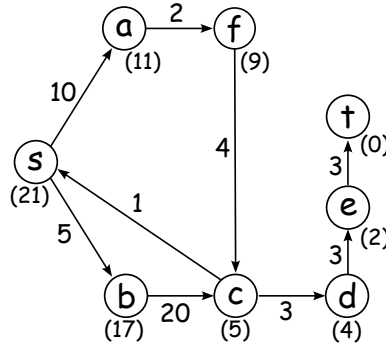
Solution.

index	state	parent	priority f	status
0	s	—	0	x
1	a	0	10	x
2	b	0	5	x
3	c	2	25	sk
4	f	1	12	x
5	c	4	16	x
6	d	5	19	x
7	e	6	22	x
8	t	7	25	t

Path found: $s \rightarrow a \rightarrow f \rightarrow c \rightarrow d \rightarrow e \rightarrow t$

Question 4. A* search with consistent heuristic (10 points)

Trace the execution of the A* search algorithm on the weighted graph below. The heuristic value for each node is given below the node in parentheses. You must write your output in the format discussed in class.



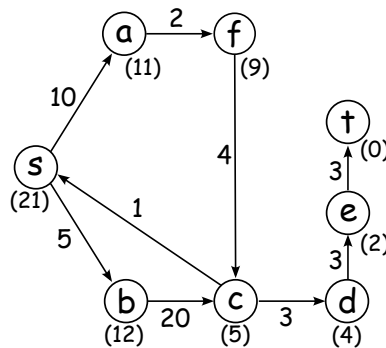
Solution.

index	state	parent	cost g	heuristic h	priority f	status
0	s	—	0	21	21	x
1	a	0	10	11	21	x
2	b	0	5	17	22	x
3	f	1	12	9	21	x
4	c	3	16	5	21	x
5	d	4	19	4	23	x
6	e	5	22	2	24	x
7	t	6	25	0	25	t

Path found: $s \rightarrow a \rightarrow f \rightarrow c \rightarrow d \rightarrow e \rightarrow t$

Question 5. A* search with admissible heuristic (10 points)

Trace the execution of the A* search algorithm on the weighted graph below, which is identical to the previous graph except for the heuristic value of b . You must write your output in the format discussed in class.



Solution.

index	state	parent	cost g	heuristic h	priority f	status
0	s	—	0	21	21	x
1	a	0	10	11	21	x
2	b	0	5	12	17	x
3	c	2	25	5	30	
4	f	1	12	9	21	x
5	c	4	16	5	21	x
6	d	5	19	4	23	x
7	e	6	22	2	24	x
8	t	7	25	0	25	t

Path found: $s \rightarrow a \rightarrow f \rightarrow c \rightarrow d \rightarrow e \rightarrow t$

Question 6. Admissible and consistent heuristics (5 points)

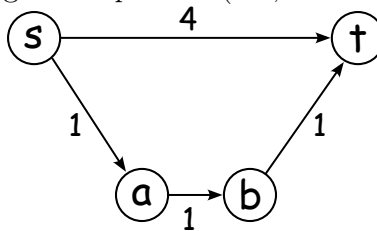
In the previous graph, Assume that we fix all values of the heuristic except for $h(b)$. Give the range of values for $h(b)$ which could make the heuristic (a) consistent and (b) admissible.

Solution.

- (a) For consistency of $s \rightarrow b$ we need $21 \leq 5 + h(b)$, so $h(b) \geq 16$. For consistency of $b \rightarrow c$ we need $h(b) \leq 20 + 5$, so $h(b) \leq 25$. Hence $16 \leq h(b) \leq 25$.
- (b) For admissibility we need $h(b) \leq$ distance from b to t , i.e., $h(b) \leq 29$.

Question 7. Waiting for target expansion (10 points)

Trace the execution of the uniform cost search algorithm on the weighted graph below. Note that the algorithm does not stop when the target is first *discovered* (i.e., added to the frontier). The algorithm terminates when the target is *expanded* (i.e., removed from the frontier).



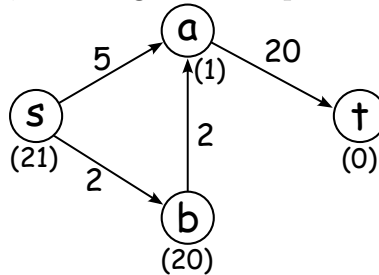
Solution.

index	state	parent	priority f	status
0	s	—	0	x
1	a	0	1	x
2	t	0	4	
3	b	1	2	x
4	t	3	3	t

Path found: $s \rightarrow a \rightarrow b \rightarrow t$

Question 8. Re-expansion: tree search vs graph search (20 points)

This question asks you to trace the execution of the A* search algorithm on the graph below, using two different variants: A* graph search and A* tree search. The heuristic given here is admissible but not consistent. One consequence of this is that A* *graph* search may fail to find the optimal path, whereas A* *tree* search is still guaranteed to find the optimal path. Why? It's possible that a node may need to be *re-expanded* if it gets added to the frontier with a lower priority than previously. In this example, node *a* gets re-expanded by tree search—but that re-expansion is missed by the graph search variant, resulting in a suboptimal path.



(a) **Graph search.** Trace the execution of the A* graph search algorithm on the graph above. Verify that the path found is suboptimal.

Solution.

index	state	parent	cost g	heuristic h	priority f	status
0	s	—	0	21	21	x
1	a	0	5	1	6	x
2	b	0	2	20	22	x
3	t	1	25	0	25	t

Path found: $s \rightarrow a \rightarrow t$

(b) **Tree search.** Trace the execution of the A* tree search algorithm on the graph above. Verify that the path found is optimal.

Solution.

index	state	parent	cost g	heuristic h	priority f	status
0	s	—	0	21	21	x
1	a	0	5	1	6	x
2	b	0	2	20	22	x
3	t	1	25	0	25	
4	a	2	4	1	5	x
5	t	4	24	0	24	t

Path found: $s \rightarrow b \rightarrow a \rightarrow t$

Question 9. NumPy and PyTorch vectors (10 points)

- (a) Write NumPy code that creates a column vector called `x` with the elements 7.5, 2, and -4 .
- (b) What is the output of `print(x.shape)`?
- (c) Write PyTorch code that creates a tensor called `y` that is a 1D-shaped vector containing the elements 26 and 34.
- (d) What is the output of `print(y.size())`?

Solution.

(a)

```
1 x = np.array([
2     [7.5],
3     [ 2],
4     [-4],
5 ])
```

(b)

```
1 (3, 1)
```

(c)

```
1 y = torch.tensor([26, 34])
```

(d)

```
1 torch.Size([2])
```

Total number of points: **95**